



U.S. General Services
Administration



Agentic AI Book Club - Week 9

Reviewing Chapter 2.3 & 2.4

AI Community of Practice - 2026

Agenda

- Housekeeping
 - End of section take-home test
 - Chapter review + Quiz review
 - Intro to next week's chapters
- 

Housekeeping

- **Agentic AI is an early and rapidly evolving knowledge domain**
- Infographics for later chapters were added -
https://github.com/GSA/MasteringAgenticAISystems_supplementals/tree/master/figures
- CPEs will be available starting with Cohort 1 - Section 2 (part 3 to 6). Specific details will be communicated in promotional materials
- First two Nvidia Certified Professional in Agentic AI test takers in early July !!

End-of-section Take-home Test

An Important milestone. The content involves:

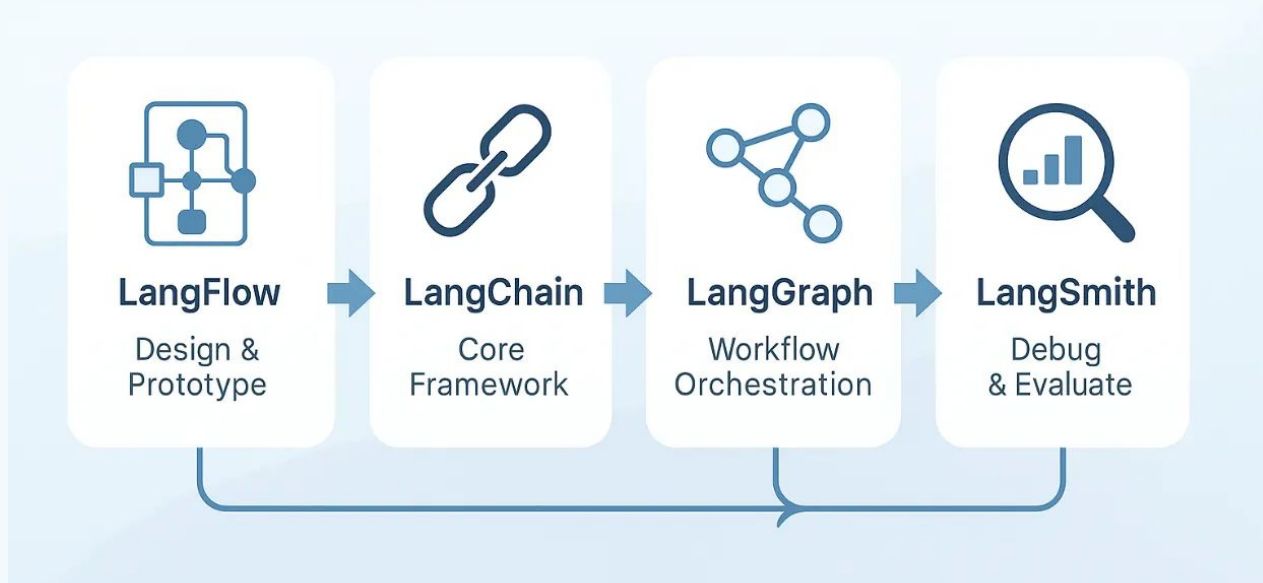
- Analyze 6 real-life agentic AI use cases (based on Federal AI Use Case Inventory 2025)
 - Grade the complexity of each use cases on categories of **Perception, Memory, Orchestration, Reasoning, Tool calling, Integration, Error handling, and Resilience**
 - Identify the ranked 3 most complex use cases, submit the rated results to a google form
- 

End-of-section Take-home Test

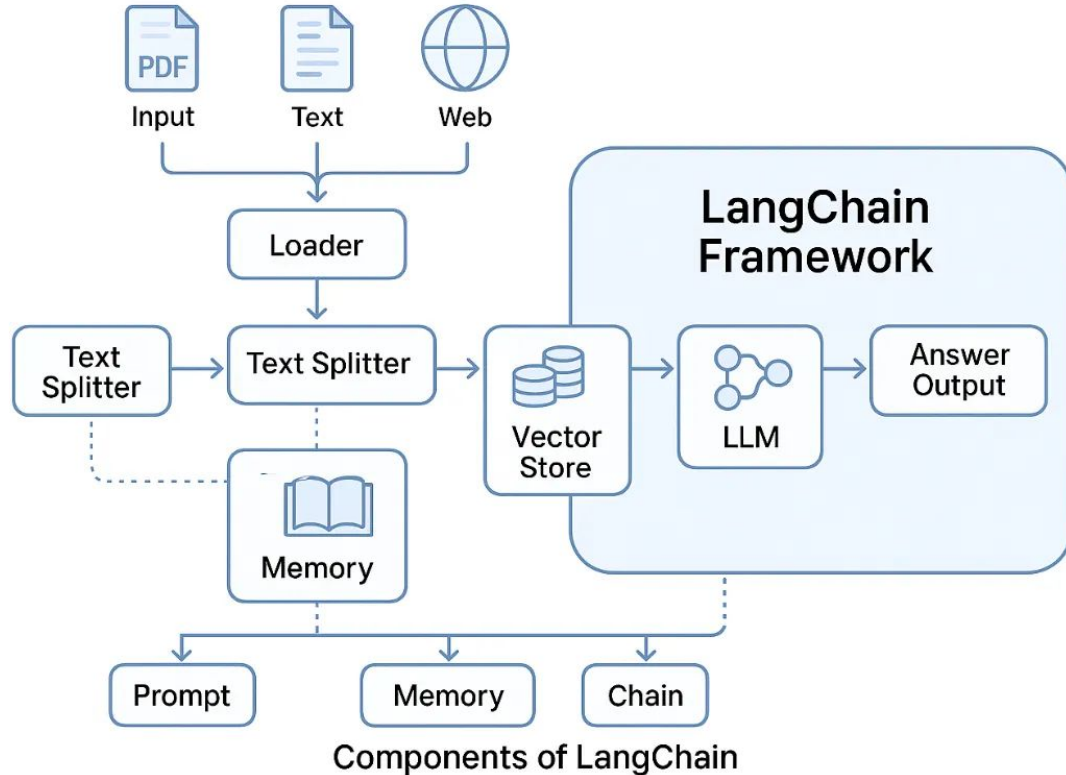
How to prepare:

- Review [relevant chapter sections](#)
 - Build your own evaluation dimensions for each category
 - Based on the dimensions, define what constitutes “Simple”, “Somewhat complex”, “Moderately complex”, “Very complex”, and “Extremely complex”
 - Spend time on understanding the details of each cases. Evaluation should be grounded in case details (to be provided next week)
- 

Langchain



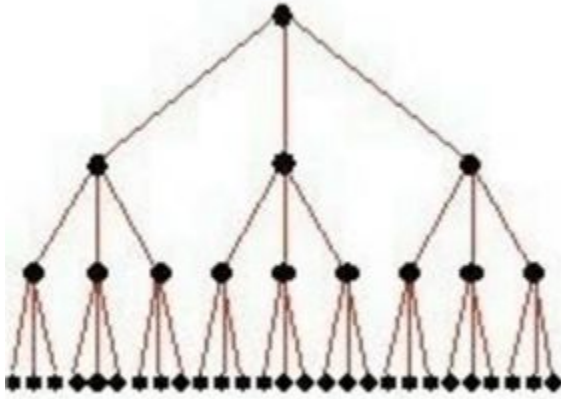
The Sequential Pattern (core LangChain)



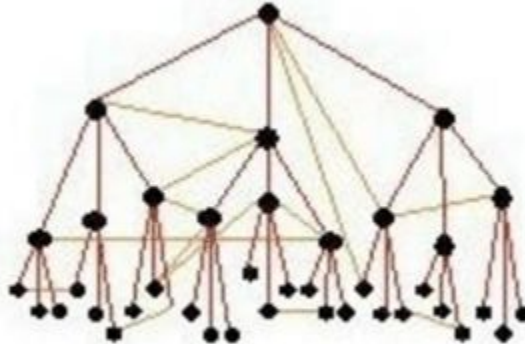
When You Have Outgrown LangChain

- Signal 1: You need iteration loops (generate-validate-retry)
 - Signal 2: Complex state beyond conversation history
 - Signal 3: Conditional branching based on tool results
 - Signal 4: Structured debugging with state inspection
 - Forcing LangChain past its limits creates technical debt
- 

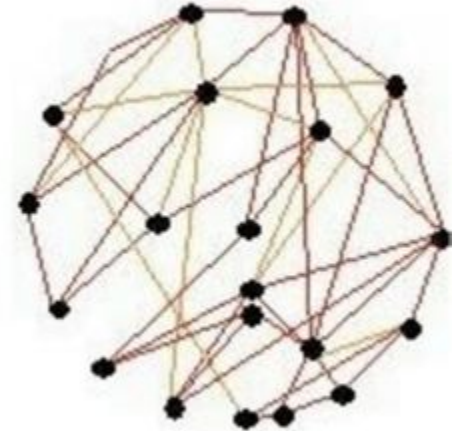
Organizational Structures



Hierarchy



Hybrid



Peer-2-Peer

When will Hierarchy not work?

Two Philosophies of Multi-Agent Coordination

Autogen

Air Traffic Control for AI Agents



Core (Actor Runtime)
"Air traffic control tower"



AgentChat
"Flight operations center"



Extensions
"Airport services"



AutoGen Studio
"Supervisor's dashboard"

CrewAI

Film Production Crew



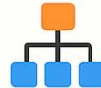
Agents
"Crew members"



Tasks
"Scenes"

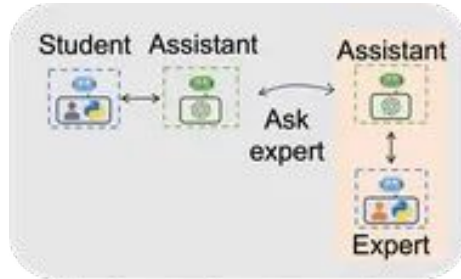


Crew (Orchestrator)
"Assistant Director"

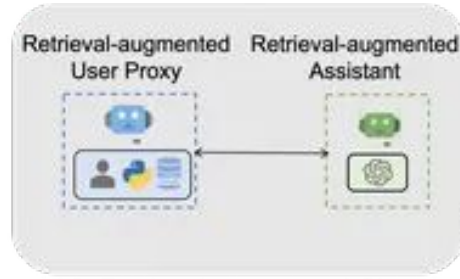


Flows
"Shooting schedule"

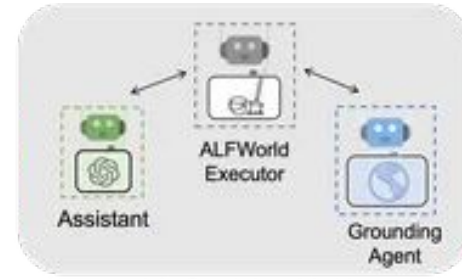
AutoGen -- Agents as Conversational Partners



A1. Math Problem Solving



A2. Retrieval-augmented Chat



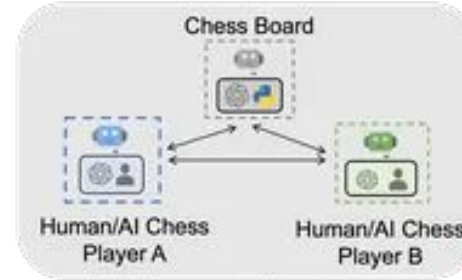
A3. Decision Making



A4. Multi-agent Coding



A5. Dynamic Group Chat



A6. Conversational Chess

AutoGen Strengths and Trade-offs

Pros

- Agents can challenge, negotiate, and refine outputs
- Transparent reasoning via full message history

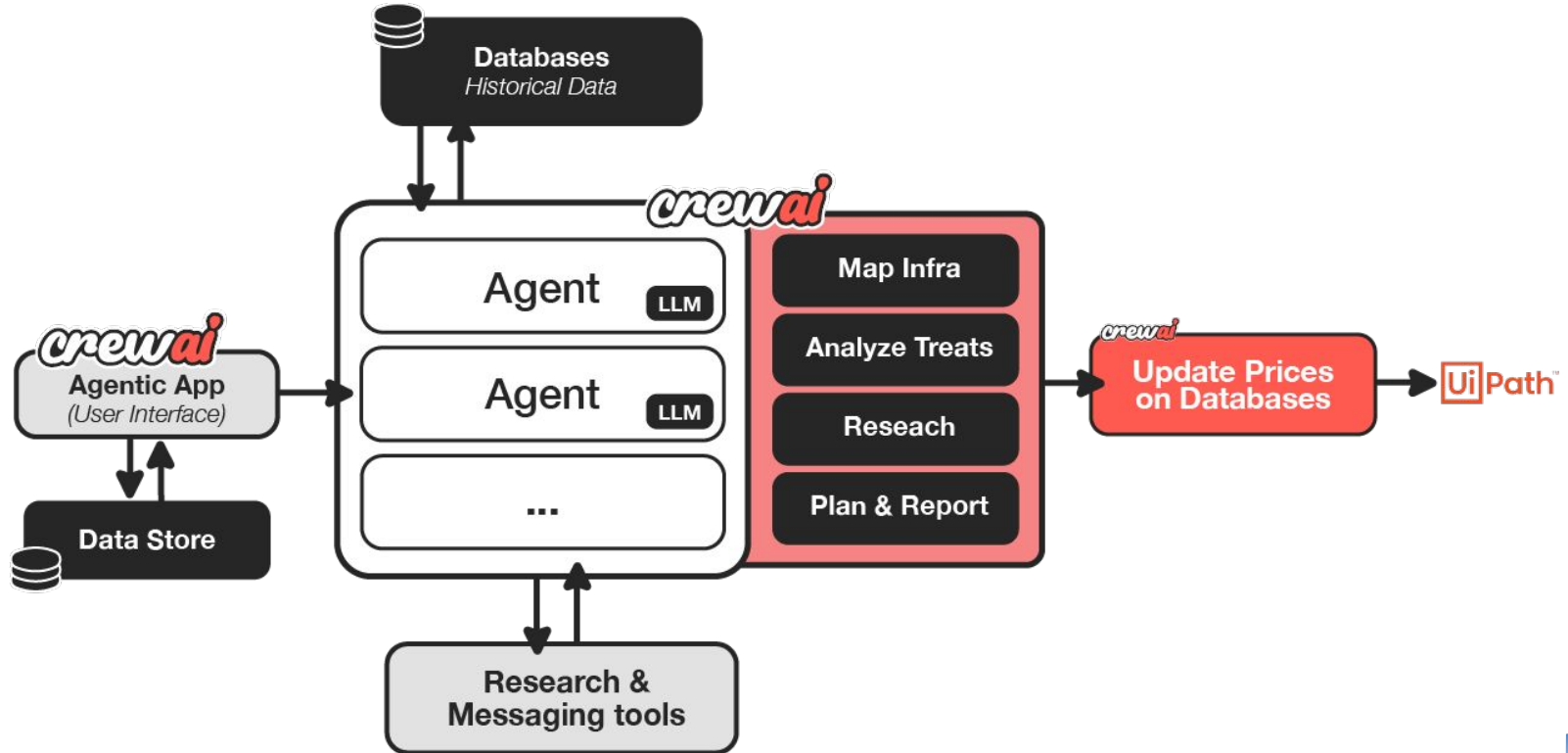
Cons

- Non-determinism complicates debugging and reproducibility
- State synchronization needs explicit management

Question to ponder

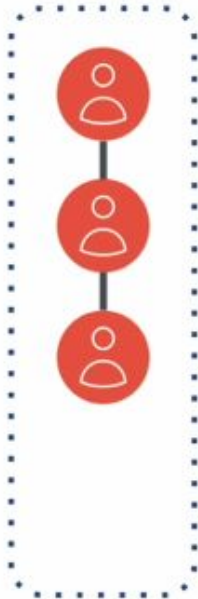
- When is non-determinism a desired properties?
 - Which use cases require agents to challenge and negotiate with each other?
- 

CrewAI -- Teams with Organizational Structure

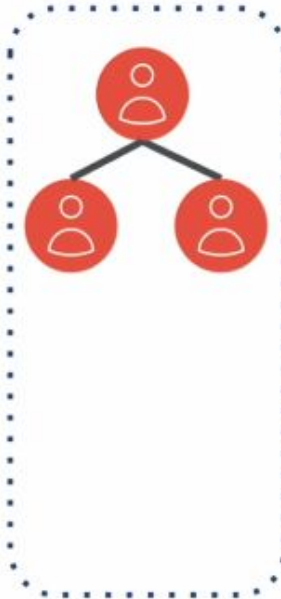


CrewAI Sequential and Hierarchical Processes

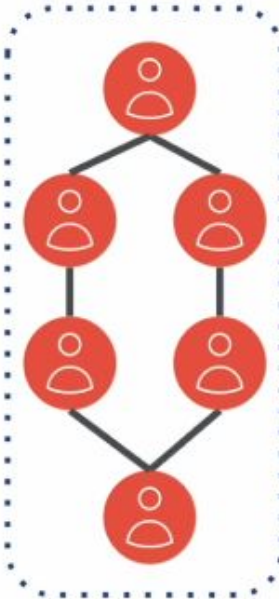
Sequential



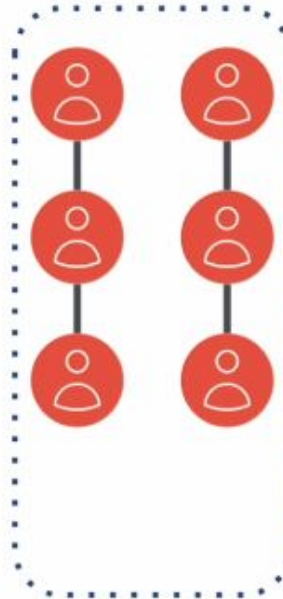
Hierarchical



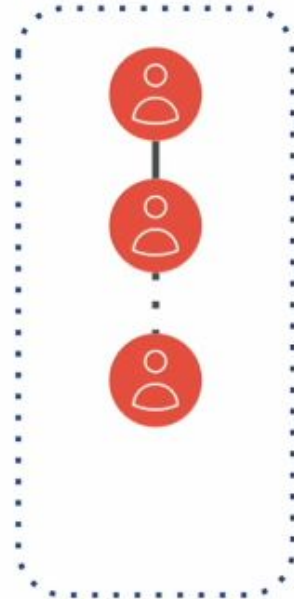
Hybrid



Parallel



Async



CrewAI Sequential and Hierarchical Processes

Scalability Factor	CrewAI (Standard)	CrewAI Flows / Enterprise
Messaging	Sequential / Hierarchical task-passing ¹	Event-driven and Asynchronous (kickoff_async) ²
State	In-memory only (accumulated task outputs)	Persistent state supported via @persist (SQLite/Database)
Distributed	Single-process execution ³	Managed Worker Nodes and horizontal scaling
Observability	Basic console logging and verbose modes	Integrated Tracing (CrewAI Tracing) and Audit Logs
Cross-Language	Python only	Python-first with native MCP and A2A protocol support

CrewAI Strengths and Trade-offs

Pros:

- Predictable execution suits production deployments
- Role clarity prevents task confusion and misrouting

Cons:

- Struggles with frequent ad-hoc agent interaction
 - Overlapping role definitions cause delegation errors
- 

Peer-2-Peer vs. Hierarchy -- Decision Framework

- Peer-2-Peer: flexible dialogue, exploration, prototyping
- Hierarchy: structured roles, sequential pipelines, production
- Consider team expertise and mental models
- Neither is universally superior -- match to workflow
- Non-determinism requires careful testing strategies
- Message history enables auditing but grows quickly
- Agent role definitions are load-bearing in production
- Start simple -- add agents only when complexity demands it

Hierarchical Agent Swarm - Example

- https://github.com/GSA-TTS/devCrew_s1
 - DevCrew_s is the specification repository for the DevCrew multi-agent framework. This repository contains all agent specifications, protocol definitions, and architectural documentation that define how autonomous AI agents collaborate to perform complex software development tasks across various domains.
- 

Quiz review

Quiz Review

You are implementing a production customer service agent with these requirements: (1) consistent, deterministic tool selection for compliance, (2) conversation memory optimized for chat models, (3) graceful handling of tool failures and parsing errors without crashes, (4) proper production logging, (5) tools that guide the agent toward correct selection decisions. Which configuration correctly satisfies all five requirements?

A:

- LLM with **temperature=0**; comprehensive tool descriptions with use cases, input formats, and examples; ConversationBufferMemory(return_messages=True); AgentExecutor(verbose=False, handle_parsing_errors=True); tools return descriptive error strings rather than raising exceptions.

Note:

- <https://docs.aws.amazon.com/nova/latest/userguide/prompting-tools-function.html>
 - <https://docs.cloud.google.com/vertex-ai/generative-ai/docs/multimodal/function-calling>
- 
- A blue decorative graphic consisting of several overlapping rectangular shapes of varying shades of blue, located in the bottom right corner of the slide.

Quiz Review

Your team is debating LangChain vs LangGraph for a customer service chatbot. The workflow is mostly linear (search knowledge base → retrieve articles → generate answer), but occasionally needs to validate generated answers and retry if quality is insufficient. The senior engineer advocates for LangGraph for "future flexibility." Evaluate the trade-offs across development velocity, maintainability, capability needs, and the occasional retry requirement. What is the most balanced recommendation?

A:

- Start with LangChain given the predominantly linear workflow; handle the occasional validation retry with a simple outer loop in the MVP; monitor retry frequency and complexity, and migrate to LangGraph only if retry logic grows frequent or complex enough to justify the framework overhead.

Note:

- <https://docs.langchain.com/oss/python/concepts/products>
- <https://www.geeksforgeeks.org/software-engineering/what-is-yagni-principle-you-arent-gonna-need-it/>

Next Chapters

- **Chapter 2.5: Semantic Kernel**

Semantic Kernel provides a central orchestration component managing service registration, plugin discovery, and execution coordination through dependency injection patterns. The framework distinguishes between semantic functions (LLM-powered reasoning) and native functions (deterministic code), enabling plugins to combine AI capabilities with reliable system integration while supporting dynamic routing through LLM-driven orchestration.

- **Chapter 2.6: Tool Integration and Function Calling**

Tool integration establishes how language models request external tool execution through structured function calling, with applications responsible for parsing JSON, validating inputs, and managing execution. The chapter covers function calling mechanics, schema design using JSON Schema standards, tool chaining for sequential dependencies, parallel execution optimization, and production considerations including error handling and NVIDIA NIM optimizations.